Microservices Architecture

Individual Assignment — 2026, Autumn

Lab Equipment Booking & Maintenance Coordination

Technical Report

Student Name: [INSERT YOUR FULL NAME]

Student ID: [INSERT YOUR STUDENT ID]

Module Code: [INSERT MODULE CODE, IF ANY]

Submission Date: [INSERT SUBMISSION DATE]

2.1 Application Overview

Problem being addressed

University labs need equipment (oscilloscopes, 3D printers, microscopes, etc.) to be booked by students and researchers, while lab administrators need equipment availability to reflect real maintenance status rather than being tracked separately in a spreadsheet. This system connects the booking workflow to the equipment/maintenance record so that a faulty piece of equipment reported at the end of a booking automatically becomes unavailable for future bookings until it is repaired.

Responsibilities of each domain microservice

Booking Service (Service A) owns the booking workflow: creating, reading, completing, cancelling and deleting bookings, and issuing JWTs for authentication (register/login). It is the primary business workflow entry point for students.

Equipment Service (Service B) owns the equipment catalog and maintenance records: full CRUD on equipment, and maintenance record creation/status updates. It is the supporting resource Booking Service depends on.

Relationship between the domain entities

A Booking references an Equipment by ID. Before a booking is confirmed, Booking Service synchronously asks Equipment Service whether that equipment is AVAILABLE. When a booking is completed and the equipment is reported faulty, Booking Service asynchronously publishes a MaintenanceRequested event; Equipment Service consumes it, flips the equipment's status to UNDER_MAINTENANCE, and opens a new MaintenanceRecord referencing the equipment and the originating booking.

Rationale for the chosen domain

This domain was chosen over a generic orders/products system because it has a genuine, asymmetric two-service relationship with a natural synchronous dependency (booking must check current equipment status) and a natural asynchronous side effect (maintenance reporting), rather than two services that could just as easily be one. It also

gave a believable reason for two different role-based authorisation patterns (students book; only lab admins manage the equipment catalog and delete bookings).

Confirmation that the domain differs from previous group work

[Insert one sentence here identifying your previous group project's domain, to confirm this is different, e.g.: "Our group project was an e-commerce order/inventory system; this individual assignment uses an unrelated university lab equipment domain."]

2.2 Architecture Overview

The diagram below shows all required components: the API Gateway as the sole client entry point, Eureka for service discovery, the Config Server for externalised configuration, the two domain microservices with their dedicated databases, RabbitMQ as the messaging infrastructure, and Zipkin as the tracing infrastructure. Solid black arrows are synchronous REST calls; dashed orange arrows are the asynchronous RabbitMQ event; thin dotted grey lines represent the cross-cutting infrastructure concerns (service registration, config fetch, trace export) common to all three application services.

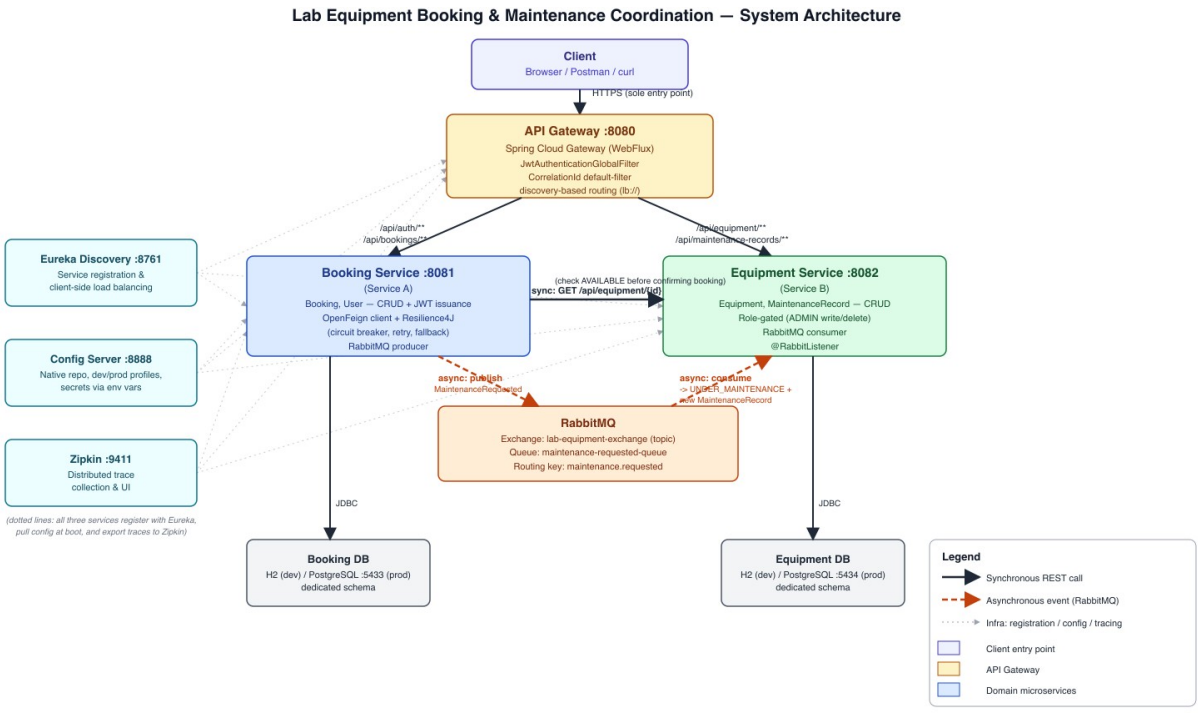


Figure 1: System architecture, synchronous and asynchronous communication paths

Responsibilities of each component

Component	Responsibility
API Gateway (:8080)	Sole client entry point. Discovery-based routing to booking-service and equipment-service. Enforces JWT authentication via a global filter before forwarding any request. Applies a CorrelationId filter to every route.
Eureka Discovery Server (:8761)	Service registry. All three application services (Gateway, Booking, Equipment) register on startup and are resolved via client-side load balancing (lb://) rather than hardcoded hosts/ports.
Config Server (:8888)	Serves externalised configuration (native file-based repo) to every client, with dev and prod profiles per service. Sensitive values (JWT secret, DB passwords) are environment-variable placeholders, never hardcoded.
Booking Service (:8081)	Owns Booking and User entities. Issues JWTs. Synchronously calls Equipment Service (OpenFeign + Resilience4J) to check availability. Publishes the MaintenanceRequested event to RabbitMQ.
Equipment Service (:8082)	Owns Equipment and MaintenanceRecord entities. Consumes the MaintenanceRequested event and updates its own state accordingly.
RabbitMQ	Messaging infrastructure. Topic exchange lab-equipment-exchange, queue maintenance-requested-queue, routing key maintenance.requested.
Zipkin (:9411)	Collects and visualises distributed traces exported via Micrometer Tracing + OpenTelemetry from all three application services.
Booking DB / Equipment DB	Dedicated database per service (H2 in-memory for the dev profile, separate PostgreSQL instances on :5433/:5434 for the prod profile) - no shared schema between services.

Significant architectural choices

The overall decomposition follows established microservices design principles for service boundaries and inter-service communication [1]. Three further significant choices are documented as full ADRs in Section 2.4: enforcing authentication centrally at the Gateway rather than in each service (ADR-001), using asynchronous messaging for the maintenance-reporting side effect rather than a second synchronous call (ADR-002), and wrapping the synchronous Booking→Equipment call with Resilience4J circuit breaker/retry rather than leaving it unprotected (ADR-003).

2.3 Implementation Evidence

All evidence below was captured live against the running system (all five services, RabbitMQ, and Zipkin, started per the README) rather than reconstructed after the fact. Raw curl transcripts are used for REST evidence (request URL, method, payload, response body, and status code are all directly visible and precise) alongside dashboard screenshots for infrastructure state.

REST APIs, Authentication and Role-Based Access

The screenshots below (rendered from real curl transcripts captured against the running system) walk through: login issuing a JWT (200); an unauthenticated request rejected by the Gateway before reaching any domain service (401); a correctly-authenticated USER rejected for an ADMIN-only write (403); CREATE (201); READ (200); UPDATE (200); a validation error (400, missing required fields); DELETE (204); and the resulting 404 after deletion. Every request shows the URL, method, payload where applicable, response body, and status code. Corresponding screencast timestamp: 09:52-11:12.

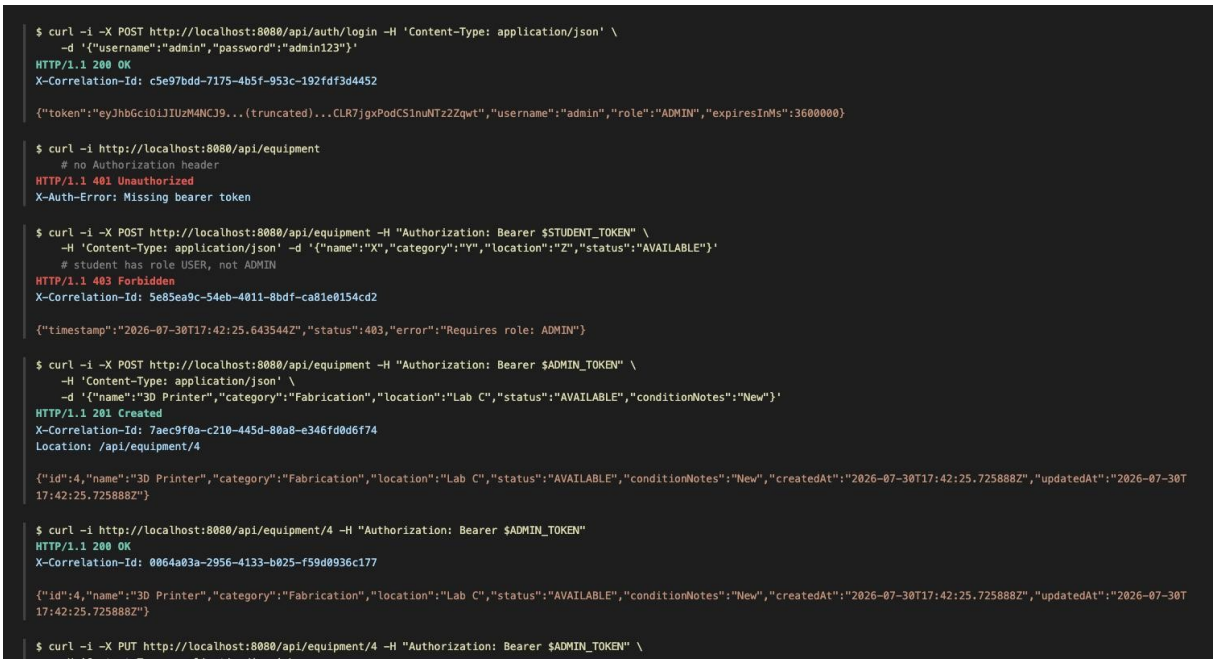


Figure 2: Login, 401, 403, CREATE, and READ

```
17:42:25.725888Z"}

$ curl -i http://localhost:8080/api/equipment/4 -H "Authorization: Bearer $ADMIN_TOKEN"
HTTP/1.1 200 OK
X-Correlation-Id: 0064a03a-2956-4133-b025-f59d0936c177

{"id":4,"name":"3D Printer","category":"Fabrication","location":"Lab C","status":"AVAILABLE","conditionNotes":"New","createdAt":"2026-07-30T17:42:25.725888Z","updatedAt":"2026-07-30T
17:42:25.725888Z"}

$ curl -i -X PUT http://localhost:8080/api/equipment/4 -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H 'Content-Type: application/json' \
    -d '{"name":"3D Printer","category":"Fabrication","location":"Lab C","status":"UNDER_MAINTENANCE","conditionNotes":"Nozzle jammed"}'
HTTP/1.1 200 OK
X-Correlation-Id: 33d8fd99-81ad-4ece-a8e6-fd153ec1cbab

{"id":4,"name":"3D Printer","category":"Fabrication","location":"Lab C","status":"UNDER_MAINTENANCE","conditionNotes":"Nozzle jammed","createdAt":"2026-07-30T17:42:25.725888Z","updat
edAt":"2026-07-30T17:42:42.452306Z"}

$ curl -i -X POST http://localhost:8080/api/equipment -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H 'Content-Type: application/json' -d '{"name":"","category":"","location":""}'
    # missing required fields
HTTP/1.1 400 Bad Request
X-Correlation-Id: b351867d-cfc9-45f5-9fde-86d9d528aaf3

{"timestamp":"2026-07-30T17:42:42.681224Z","status":400,"error":"Validation Failed","fieldErrors":{"name":"name is required","status":"status is required","category":"category is req
uired","location":"location is required"}}

$ curl -i -X DELETE http://localhost:8080/api/equipment/4 -H "Authorization: Bearer $ADMIN_TOKEN"
HTTP/1.1 204 No Content
X-Correlation-Id: 7de2ca76-6ba2-4c65-90f6-53ec291dc53b

$ curl -i http://localhost:8080/api/equipment/4 -H "Authorization: Bearer $ADMIN_TOKEN"
    # deleted above
HTTP/1.1 404 Not Found
X-Correlation-Id: ee53a1fd-393b-4362-9dbc-1165883acdb9

{"timestamp":"2026-07-30T17:42:43.827709Z","status":404,"error":"Equipment 4 not found"}
```

Figure 3: READ, UPDATE, validation error (400), DELETE, and 404

Gateway, Service Discovery and Configuration

All three application services (API-GATEWAY, BOOKING-SERVICE, EQUIPMENT-SERVICE) are shown below registered and UP in the Eureka dashboard. Corresponding screencast timestamp: 08:50-09:20.

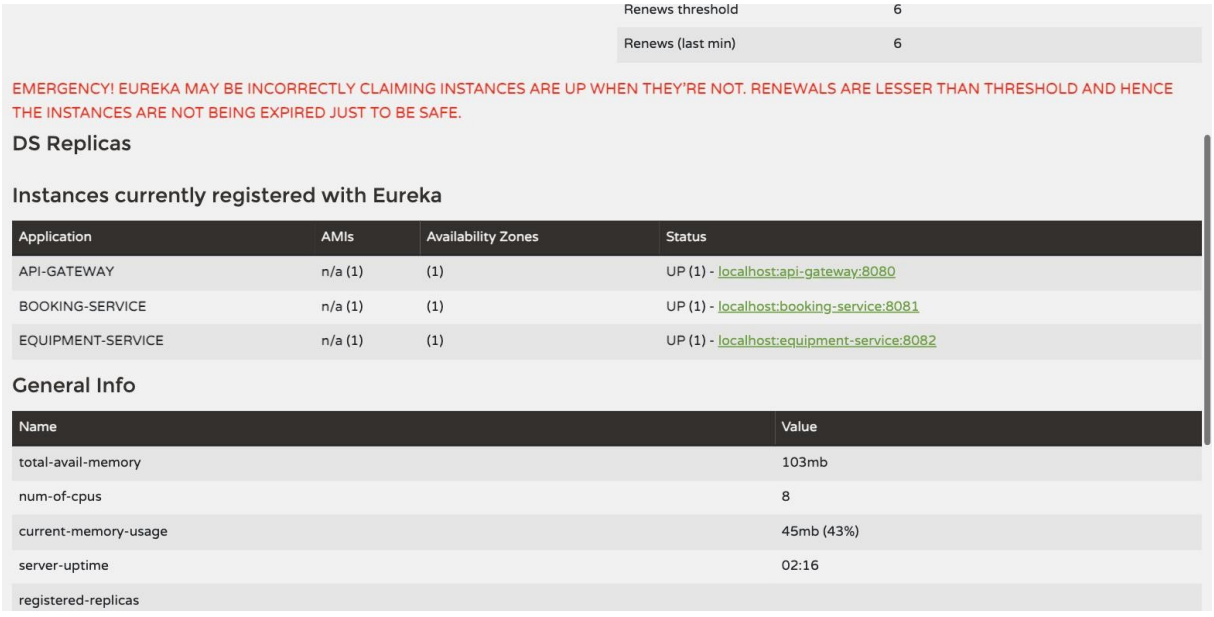


Figure 4: Eureka dashboard — all three services registered and UP

Configuration is fetched from the Config Server at boot. The two screenshots below query the same service (booking-service) under the dev and prod profiles directly from the Config Server: dev serves an H2 in-memory datasource with DEBUG-level logging; prod serves a PostgreSQL datasource with credentials as unresolved environment-variable placeholders (${BOOKING_DB_HOST}, ${BOOKING_DB_PASSWORD}, etc. - never hardcoded values) and WARN-level logging. Corresponding screencast timestamp: 09:20-09:52.

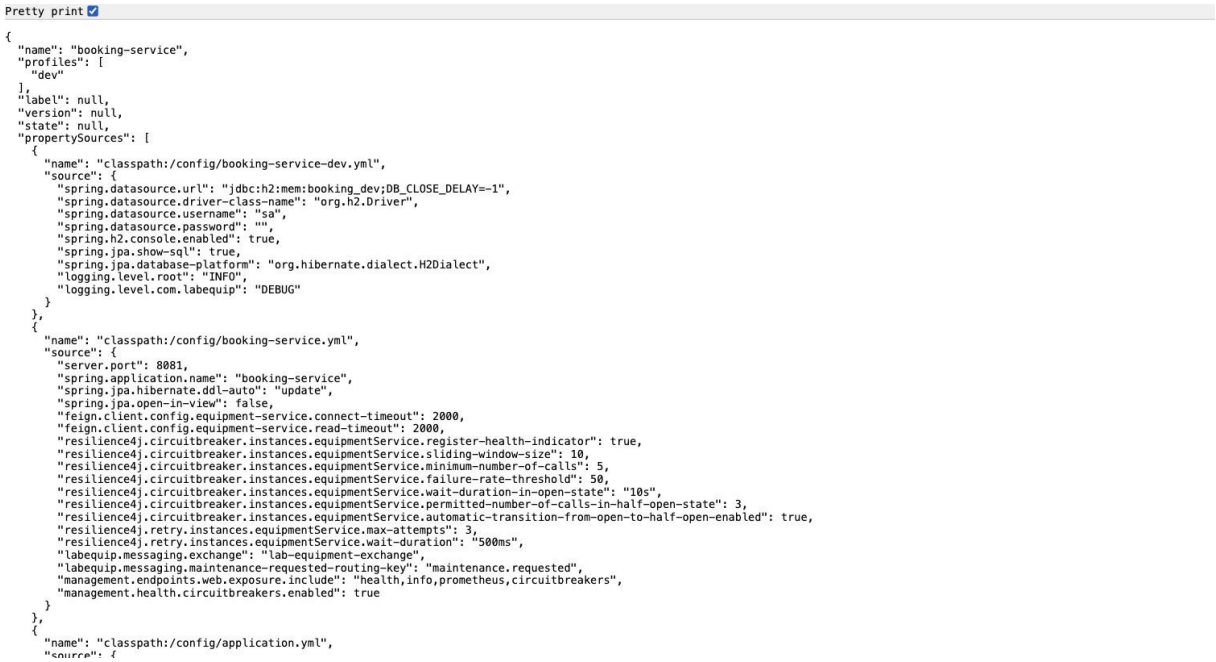
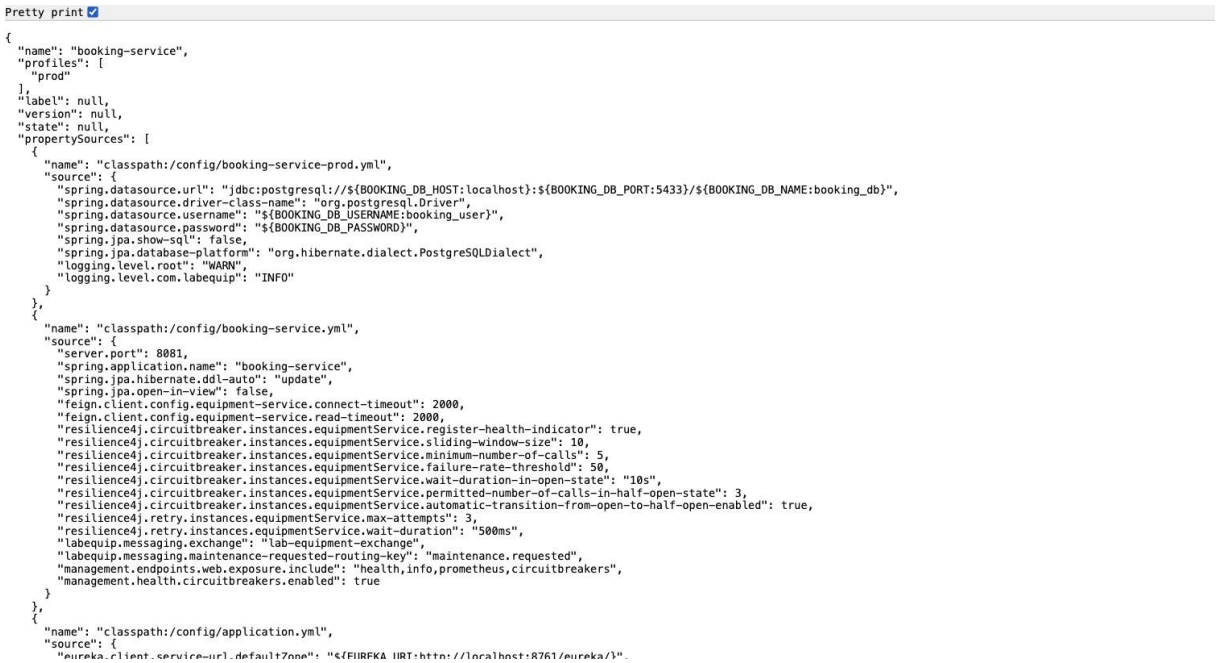


Figure 5: Config Server — booking-service, dev profile (H2, DEBUG)



Pretty print


```
{
  "name": "booking-service",
  "profiles": [
    "prod"
  ],
  "label": null,
  "version": null,
  "state": null,
  "propertySources": [
    {
      "name": "classpath:/config/booking-service-prod.yml",
      "source": {
        "spring.datasource.url": "jdbc:postgresql://${BOOKING_DB_HOST:localhost}:${BOOKING_DB_PORT:5433}/${BOOKING_DB_NAME:booking_db}",
        "spring.datasource.driver-class-name": "org.postgresql.Driver",
        "spring.datasource.username": "${BOOKING_DB_USERNAME:booking_user}",
        "spring.datasource.password": "${BOOKING_DB_PASSWORD}",
        "spring.jpa.show-sql": false,
        "spring.jpa.database-platform": "org.hibernate.dialect.PostgreSQLDialect",
        "logging.level.root": "WARN",
        "logging.level.com.labequip": "INFO"
      }
    },
    {
      "name": "classpath:/config/booking-service.yml",
      "source": {
        "server.port": 8081,
        "spring.application.name": "booking-service",
        "spring.jpa.hibernate.ddl-auto": "update",
        "spring.jpa.open-in-view": false,
        "feign.client.config.equipment-service.connect-timeout": 2000,
        "feign.client.config.equipment-service.read-timeout": 2000,
        "resilience4j.circuitbreaker.instances.equipmentService.register-health-indicator": true,
        "resilience4j.circuitbreaker.instances.equipmentService.sliding-window-size": 10,
        "resilience4j.circuitbreaker.instances.equipmentService.minimum-number-of-calls": 5,
        "resilience4j.circuitbreaker.instances.equipmentService.failure-rate-threshold": 50,
        "resilience4j.circuitbreaker.instances.equipmentService.wait-duration-in-open-state": "10s",
        "resilience4j.circuitbreaker.instances.equipmentService.permitted-number-of-calls-in-half-open-state": 3,
        "resilience4j.circuitbreaker.instances.equipmentService.automatic-transition-from-open-to-half-open-enabled": true,
        "resilience4j.retry.instances.equipmentService.max-attempts": 3,
        "resilience4j.retry.instances.equipmentService.wait-duration": "500ms",
        "labequip.messaging.exchange": "lab-equipment-exchange",
        "labequip.messaging.maintenance-requested-routing-key": "maintenance.requested",
        "management.endpoints.web.exposure.include": "health,info,prometheus,circuitbreakers",
        "management.health.circuitbreakers.enabled": true
      }
    },
    {
      "name": "classpath:/config/application.yml",
      "source": {
        "eureka.client.service-url.defaultZone": "${EUREKA_URI:http://localhost:8761/eureka/}",
```


Figure 6: Config Server — booking-service, prod profile (PostgreSQL, env-var secrets, WARN)

Service-to-Service Communication and Resilience

Equipment Service was stopped to simulate an outage. The circuit breaker's state was queried before and after repeated failing calls, showing the transition CLOSED → OPEN (actual failure rate 80%, exceeding the 50% threshold) → automatic HALF_OPEN after the configured wait duration [2]. While OPEN, calls fail immediately without attempting the network call (notPermittedCalls), and the caller receives a clear 503 in roughly a second rather than a hang. Corresponding screencast timestamp: 11:34-12:29.

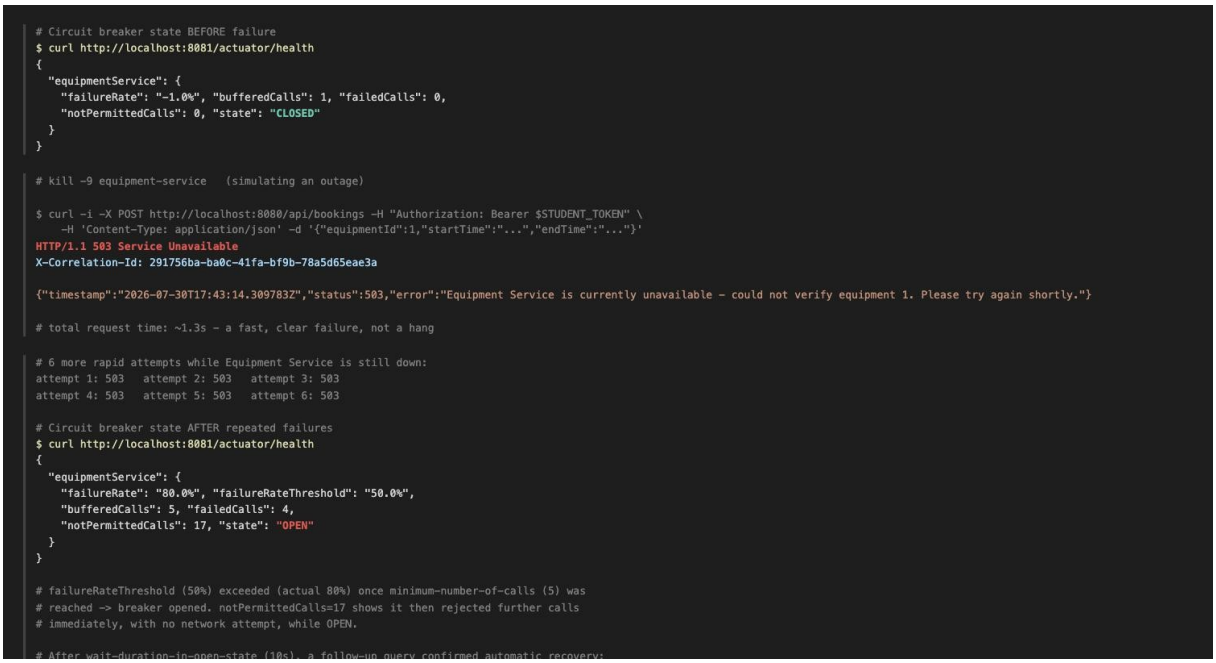


Figure 7: Resilience4J circuit breaker — CLOSED, repeated failures, OPEN, then HALF_OPEN

Asynchronous Messaging

The RabbitMQ management UI below shows the lab-equipment-exchange (topic exchange) and maintenance-requested-queue with live message rates, confirming both are declared correctly and processing traffic [3]. Corresponding screencast timestamp: 12:29-13:04.

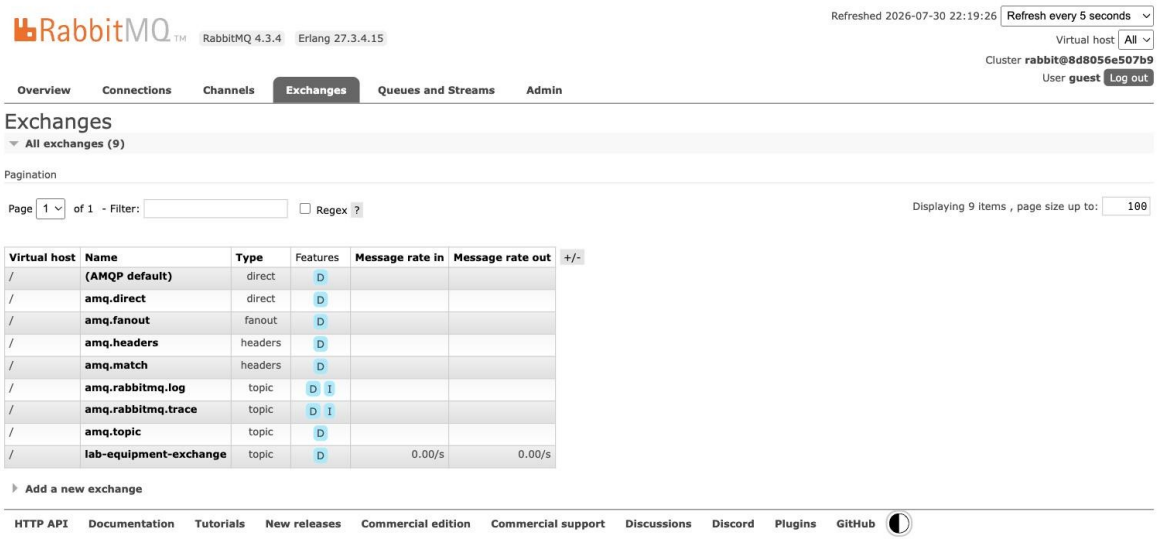


Figure 8: RabbitMQ Exchanges — lab-equipment-exchange with active message rate

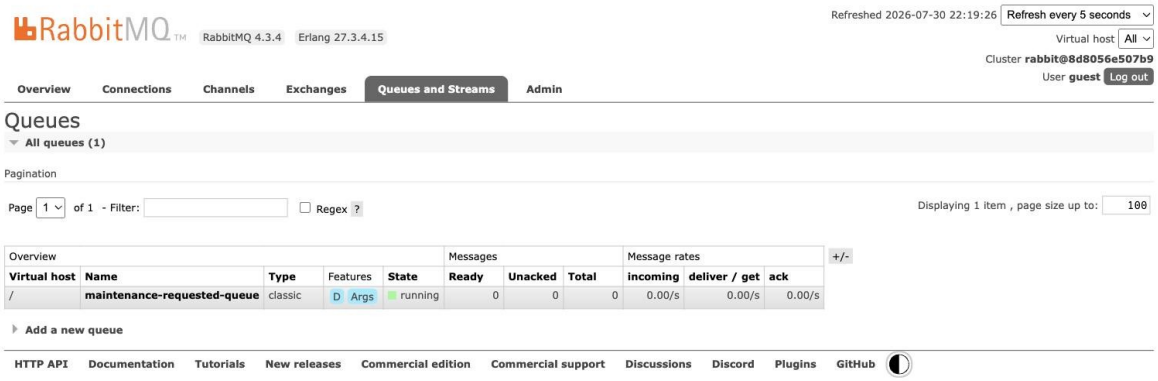


Figure 9: RabbitMQ Queues — maintenance-requested-queue

To directly evidence the event payload, Equipment Service was stopped so a published event would remain queued, and RabbitMQ's own "Get messages" inspector (Nack/requeue, so the message was not lost) was used to view it: the exchange, routing key, and full JSON body are all visible, along with a traceparent header showing the OpenTelemetry trace context is carried inside the message itself.

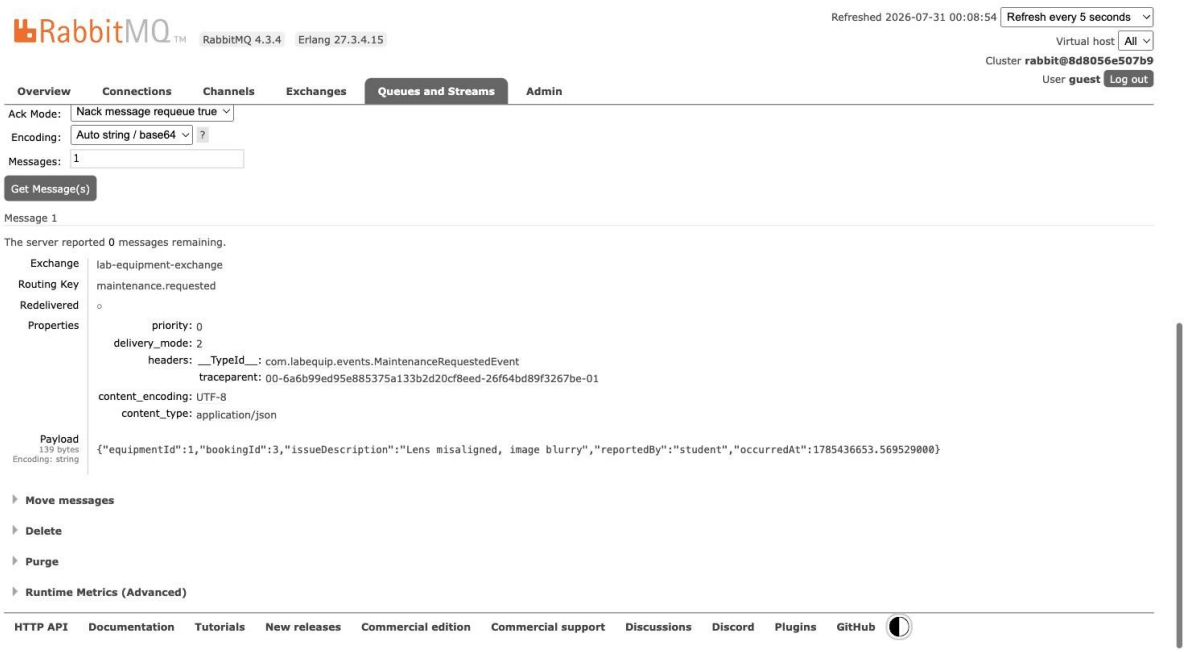


Figure 10: Actual MaintenanceRequested message payload, inspected via RabbitMQ

See ADR-002 for the justification of using asynchronous messaging rather than a synchronous call for this specific interaction.

Database State Changes

To confirm the state change shown above was actually caused by Equipment Service consuming the event (rather than a direct write), both queries below were run directly against Equipment Service's own H2 database via its console, immediately after the MaintenanceRequested event was published and consumed: the equipment row's STATUS column has flipped to UNDER_MAINTENANCE, and a new MAINTENANCE_RECORD row references the originating booking via SOURCE_BOOKING_ID - both entirely within Equipment Service's own dedicated schema. Corresponding screencast timestamp: 13:04-13:36.

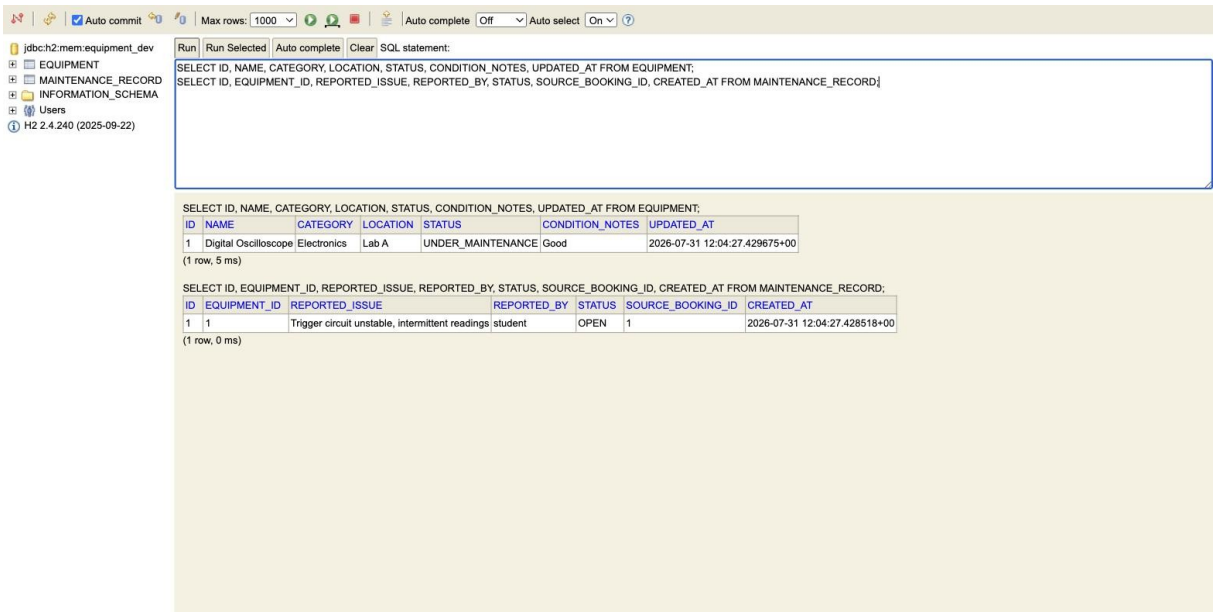


Figure 11: Equipment Service H2 console - EQUIPMENT and MAINTENANCE_RECORD rows after async event consumption

Observability and Distributed Tracing

Distributed tracing follows the pipeline pattern of propagating trace context across service and messaging boundaries [5]. The trace below shows the full synchronous chain for booking creation in one trace: the Gateway receives the request, routes to Booking Service, which in turn calls Equipment Service via OpenFeign to check availability — all three services appear under a single trace ID. Corresponding screencast timestamp: 11:12-11:34.

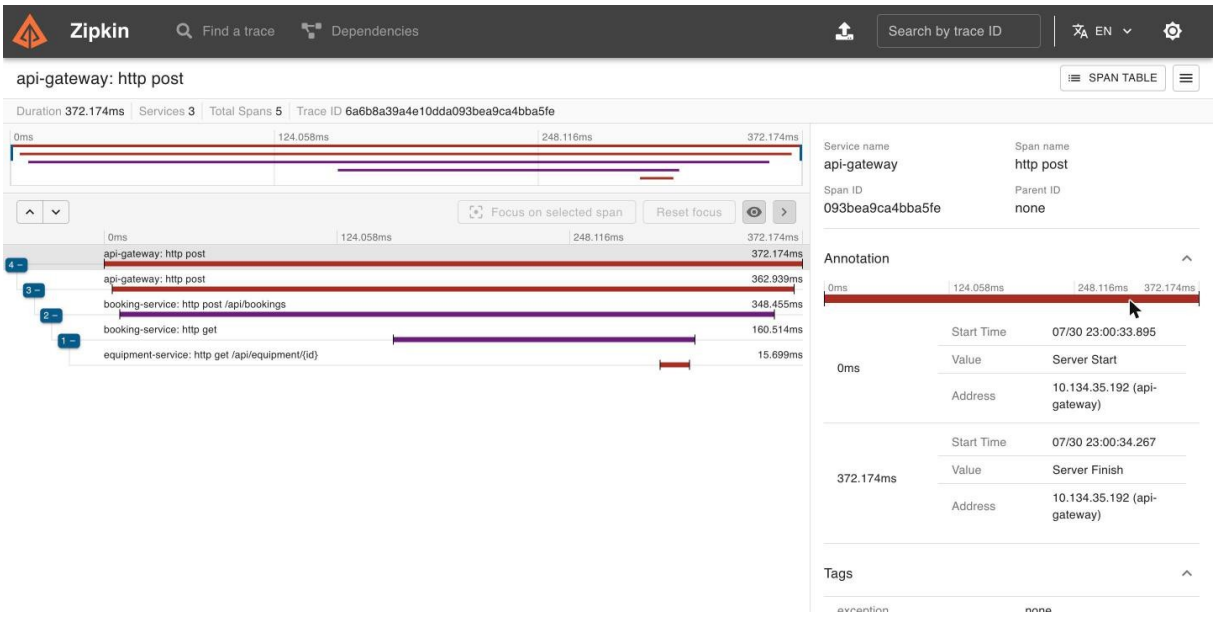


Figure 12: Zipkin trace — Gateway to Booking Service to Equipment Service (sync)

The trace below shows propagation through the asynchronous path: Booking Service's publish to lab-equipment-exchange/maintenance.requested and Equipment Service's receive from maintenance-requested-queue both appear as spans under the same trace ID as the originating booking-completion request, demonstrating trace propagation across the message broker.

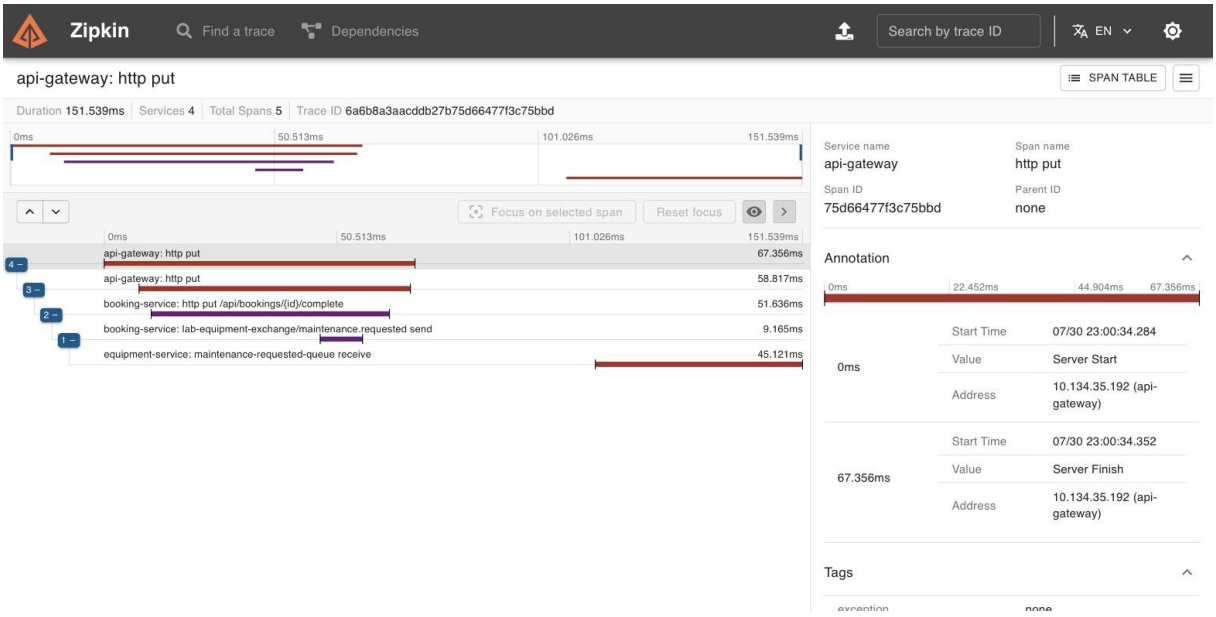


Figure 13: Zipkin trace — async publish/consume spans under one trace ID

Correlation IDs and trace/span IDs both appear in every log line via the pattern [service-name,traceId,spanId]. The screenshot below greps the same trace ID out of both booking-service.log and equipment-service.log for one async hop, showing the identical traceId shared across both services with distinct per-service span IDs.

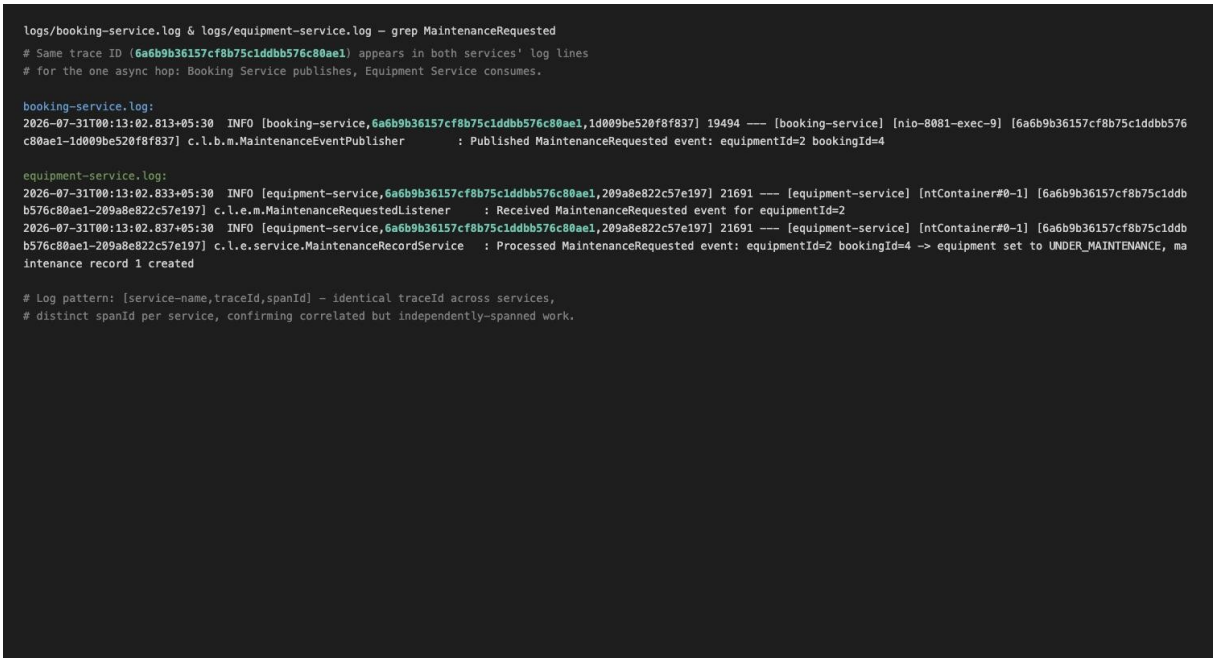


Figure 14: Matching trace ID across booking-service and equipment-service logs

2.4 Architecture Decision Records (ADRs)

Full ADRs (Context, Decision, Alternatives Considered, Consequences, Implementation Evidence) are included in Appendix A and as standalone files in docs/adr/. Summaries follow.

ADR-001: Gateway-Centred Security

Decision: authentication is enforced once, centrally, at the API Gateway (JwtAuthenticationGlobalFilter), which validates the JWT and forwards the identity via X-Auth-User/X-Auth-Roles headers; each service additionally runs a lightweight RoleGuard so a request that bypasses the Gateway is still rejected rather than trusted implicitly. This follows recommended practice for centralising authentication concerns at the gateway in Spring-based microservice deployments [4]. See Appendix A.1 for full rationale and consequences.

Alternative Considered	Why Not Chosen
Per-service Spring Security + JWT decoding	Duplicates verification logic across every service instead of once.
Trust downstream services with no re-check	Defeats the "avoid direct access" requirement - a request that bypasses the Gateway would be trusted implicitly.
OAuth2 with an external Authorization Server	Disproportionate operational cost (Keycloak/Spring Authorization Server) for a two-role system.

ADR-002: Event-Driven Communication for Maintenance Reporting

Decision: booking completion publishes a MaintenanceRequested event to RabbitMQ rather than calling Equipment Service synchronously, because the caller does not need the result, the event must survive Equipment Service being temporarily down, and this interaction's failure domain should be decoupled from booking completion's success. RabbitMQ was chosen over Kafka based on comparative message-queue benchmarking showing RabbitMQ's lower operational overhead is well suited to lower-throughput, single-event-type workloads like this one [3]. See Appendix A.2 for full rationale and consequences.

Alternative Considered	Why Not Chosen
Synchronous call mirroring the availability check	Couples booking completion's success/latency to Equipment Service's availability for a side effect the caller doesn't wait on.

Kafka instead of RabbitMQ	Unneeded complexity (partitioning, consumer groups, log replay) for one event type at this scale [1].
Spring Cloud Stream	Adds a binder abstraction layer not needed for one exchange/queue, and would obscure the wiring the report needs to evidence directly.

ADR-003: Resilience and Failure Handling for the Sync Booking to Equipment Call

Decision: the synchronous availability check is wrapped with Resilience4J Retry (3 attempts, 500ms apart) and CircuitBreaker (opens at 50%+ failure rate over a 10-call window, min. 5 calls), backed by 2s Feign connect/read timeouts, with a fallback that returns a clear 503 instead of a hang or raw exception - consistent with recent work showing a tuned circuit-breaker pattern reduces failure-detection and state-transition delay in microservice architectures [2]. See Appendix A.3 for full rationale and consequences.

Alternative Considered	Why Not Chosen
No resilience wrapper	Violates the assignment's explicit resilience requirement outright.
Retry only, no circuit breaker	Keeps hammering a genuinely-down dependency on every request instead of protecting Booking Service's own resources.
Circuit breaker only, no retry	A single transient TCP hiccup would count as a hard failure and could trip the breaker unnecessarily.
Async @TimeLimiter	Would force the synchronous booking flow onto an async (CompletableFuture) model for no real benefit over Feign's own connect/read timeouts.

2.5 Screencast Cross-Reference Table (Mandatory)

Screencast timestamps below refer to the submitted screencast recording (Screencast.mp4, total runtime 14:29).

Requirement	Section	Screencast	ADR
REST CRUD (Equipment): create/read/update/delete, status codes, validation	2.3 REST APIs	09:52-11:12	-
Eureka registration (all 3 services UP)	2.3 Gateway/Discovery/Config	08:50-09:20	-
Config Server: dev/prod profiles, externalised secrets	2.3 Gateway/Discovery/Config	09:20-09:52	-
Gateway routing + CorrelationId filter	2.2 / 2.3	09:52-11:12	ADR-001
JWT login, 401 unauth., 403 wrong role	2.3 Security	09:52-11:12	ADR-001
Sync call Booking → Equipment (OpenFeign)	2.3 Resilience	11:12-11:34	ADR-003
Resilience4J: circuit open, fallback 503	2.3 Resilience	11:34-12:29	ADR-003
Async event publish/consume (RabbitMQ)	2.3 Messaging	12:29-13:04	ADR-002
State change: UNDER_MAINTENANCE + record	2.3 Database State Changes	13:04-13:36	ADR-002
Trace: sync chain (Gateway/Booking/Equip.)	2.3 Observability	11:12-11:34	-
Trace: async chain (publish/consume spans)	2.3 Observability	13:36-14:00	ADR-002
Correlation IDs in logs	2.3 Observability	14:00-14:22	-

3. Source Code Repository

Repository link: [INSERT GIT REMOTE URL HERE - push the local repository at lab-equipment-microservices/ to GitHub/GitLab and share access with the module team].

The repository contains a clear README with build/run instructions, and demonstrates incremental development through multiple meaningful commits (parent scaffold; discovery + config server; gateway; equipment service; booking service; docker-compose + smoke test; README) rather than a single bulk upload.

References

IEEE citation style. In-text citation markers (e.g. [1]) appear throughout Sections 2.2, 2.3, and 2.4 where the corresponding source directly informed a design decision.

[1] V. Velepucha and P. Flores, "A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges," IEEE Access, vol. 11, pp. 88339-88358, 2023, doi: 10.1109/ACCESS.2023.3305687.

[2] A. Hlybovets and I. Paprotskyi, "Increasing the Fault Tolerance in Microservice Architecture," Cybernetics and Systems Analysis, vol. 60, no. 3, pp. 480-488, 2024, doi: 10.1007/s10559-024-00689-0.

[3] R. Maharjan, M. S. H. Chy, M. A. Arju, and T. Cerny, "Benchmarking Message Queues," Telecom, vol. 4, no. 2, pp. 298-312, 2023, doi: 10.3390/telecom4020018.

[4] C. L. Aldea and R. Bocu, "Authentication Challenges and Solutions in Microservice Architectures," Applied Sciences, vol. 15, no. 22, art. 12088, 2025, doi: 10.3390/app152212088.

[5] C. Colarusso, A. De Caro, I. Falco, L. Goglia, and E. Zimeo, "A distributed tracing pipeline for improving locality awareness of microservices applications," Software: Practice and Experience, vol. 54, no. 6, pp. 1118-1140, 2024, doi: 10.1002/spe.3317.

Appendix A: Full Architecture Decision Records

A.1 ADR-001: Gateway-Centred Security

Status

Accepted

Context

The system exposes two domain microservices (Booking Service, Equipment Service) behind a Spring Cloud Gateway. The assignment requires JWT-based authentication, enforced before requests reach a downstream microservice, plus role-based authorisation with at least two differently protected endpoint categories (read-only for any authenticated user; write/delete restricted to ADMIN).

A decision was needed on **where** authentication is enforced:

1. At the Gateway only, with downstream services trusting a forwarded identity header. 2. At each downstream microservice independently (e.g. Spring Security + JWT decoding in every service). 3. Duplicated in both places.

The domain has two roles only (`USER`, `ADMIN`), issued by Booking Service's `/api/auth/login` and `/api/auth/register` endpoints (`AuthController`, `JwtTokenProvider`), signed with a shared HMAC secret sourced from Config Server (`jwt.secret`, environment-variable backed, never hardcoded).

Decision

Authentication is enforced centrally at the API Gateway via `JwtAuthenticationGlobalFilter` (`api-gateway/src/main/java/com/labequip/gateway/security/JwtAuthenticationGlobalFilter.java`):

- Every request not matching `security.public-paths` (`/api/auth/**`, `/actuator/**`) must present a valid `Bearer` JWT, verified via `JwtValidator` against the shared secret.
- On success, the Gateway enriches the request with `X-Auth-User` and `X-Auth-Roles` headers before forwarding it downstream (a form of trusted-header propagation, not re-issuing a token).

- On failure (missing/invalid/expired token), the Gateway returns `401` immediately - the request never reaches Booking Service or Equipment Service.

Each downstream service additionally runs a lightweight `RoleGuard` (`equipment-service/.../security/RoleGuard.java`, `booking-service/.../security/RoleGuard.java`) that re-checks `X-Auth-User`/`X-Auth-Roles` before performing the operation:

- `EquipmentController`: `GET` requires any authenticated user; `POST`/`PUT`/`DELETE` require `ADMIN`.
- `BookingController`: `GET`/`POST`/complete/cancel require any authenticated user; `DELETE` requires `ADMIN`.

This is defence-in-depth, not duplicated authentication: the services trust the headers as an *identity assertion from a trusted network hop* (the Gateway), not as a token they independently verify. If a request reaches a service without those headers - i.e. it bypassed the Gateway - `RoleGuard.requireAuthenticated` rejects it with `401`, which also satisfies the requirement that "direct browser/client access to microservices should be avoided" by making direct access non-functional for anything but public health checks.

Alternatives Considered

Alternative Considered	Why Not Chosen
Per-service Spring Security + JWT decoding	each service independently validates the JWT and applies `@PreAuthorize`. Rejected as the primary mechanism: it duplicates JWT parsing logic across every service, means a change to the auth scheme (e.g. rotating to OAuth2) has to be made N times, and does not stop a request that reaches a service directly (bypassing the Gateway) unless each service also re-implements the check - at which point the Gateway's filter is redundant work. Full Spring Security was also heavier than needed for two roles.
Trust downstream services completely (no RoleGuard)	simplest, but means any request that reaches a service directly (e.g. hitting `localhost:8082` instead of the Gateway) would be treated as authenticated with no role, defeating the "avoid direct access" requirement. Rejected.
OAuth2 with an external Authorization Server	(Keycloak / Spring Authorization Server) - more standard for production and would support token introspection/refresh, but adds an extra moving part and setup/operational cost disproportionate to a two-role, coursework-scoped system. JWT issued directly by Booking Service was judged sufficient and simpler to demonstrate end to end within the assignment's scope.

Gateway-centred JWT enforcement with Spring Security components, as adopted here, is consistent with recent published guidance on securing Spring Boot-based microservice ecosystems [4].

Consequences

Positive

- Single point of authentication enforcement; adding a third microservice only requires adding a route + optional `RoleGuard`, not re-implementing JWT verification.
- Clean separation: Gateway answers "is this request authenticated, and as whom", services answer "is this identity allowed to do this specific operation".
- Satisfies the "at least two differently protected endpoint categories" requirement uniformly across both services (read vs. write/delete).

Negative / trade-offs

- The shared JWT secret must be identical across Gateway, Booking Service, and Equipment Service (currently distributed via Config Server's shared `application.yml`) - a key rotation requires updating one place, but restarting all three processes.
- The `X-Auth-User`/`X-Auth-Roles` header trust model assumes the network between Gateway and the domain services is not itself hostile (no mutual TLS or signed headers) - acceptable for this assignment's scope but would need hardening (e.g. mTLS or a short-lived signed hop token) in a real production deployment.
- Feign calls from Booking Service to Equipment Service must manually forward these headers (`FeignHeaderForwardingConfig`), since that hop also bypasses the Gateway.

Implementation Evidence

- `api-gateway/src/main/java/com/labequip/gateway/security/JwtAuthenticationGlobalFilter.java`
- `api-gateway/src/main/java/com/labequip/gateway/security/JwtValidator.java`
- `booking-service/src/main/java/com/labequip/booking/web/AuthController.java`
- `booking-service/src/main/java/com/labequip/booking/security/JwtTokenProvider.java`
- `equipment-service/src/main/java/com/labequip/equipment/security/RoleGuard.java`
- `booking-service/src/main/java/com/labequip/booking/config/FeignHeaderForwardingConfig.java`

- Config: `config-server/src/main/resources/config/application.yml` (`jwt.secret`), `api-gateway.yml` (`security.public-paths`)
- Report evidence: Section 2.3 "REST APIs, Authentication and Role-Based Access" (`docs/screenshots/09-crud-security-part1.jpg`: 200 login, 401 no-token, 403 wrong-role)
- Screencast timestamp: **09:52–11:12**

References

[4] C. L. Aldea and R. Bocu, "Authentication Challenges and Solutions in Microservice Architectures," *Applied Sciences*, vol. 15, no. 22, art. 12088, 2025, doi: 10.3390/app152212088.

A.2 ADR-002: Event-Driven Communication for Maintenance Reporting

Status

Accepted

Context

When a student completes a booking and reports the equipment as faulty, two things must happen: Booking Service must record the completion, and Equipment Service must be told to take the equipment out of service and open a maintenance record. The assignment requires at least one interaction in the system to use asynchronous messaging rather than a synchronous REST call, with a justification for why messaging was the better fit here.

The alternative - having `BookingService.complete()` call Equipment Service synchronously (the same way it already does for the availability check in `EquipmentAvailabilityService`) - was considered and rejected for this specific interaction.

Decision

`BookingService.complete()` (`booking-service/.../service/BookingService.java`) publishes a `MaintenanceRequestedEvent` to RabbitMQ (`MaintenanceEventPublisher`) after marking the booking `COMPLETED`, instead of calling Equipment Service directly:

- Exchange: `lab-equipment-exchange` (topic), routing key `maintenance.requested` (`booking-service/.../messaging/RabbitMQConfig.java`)
- Queue: `maintenance-requested-queue`, bound in Equipment Service (`equipment-service/.../messaging/RabbitMQConfig.java`)
- Consumer: `MaintenanceRequestedListener.onMaintenanceRequested()` calls `MaintenanceRecordService.handleMaintenanceRequested()`, which sets the equipment's status to `UNDER_MAINTENANCE` and creates a new `MaintenanceRecord` - a genuine, meaningful state change, not a no-op notification.
- The event class `com.labequip.events.MaintenanceRequestedEvent` is deliberately duplicated (same package + class name) in both services rather than shared as a library, so the two services remain independently deployable and the event is treated as a payload *contract*, not a compile-time dependency.

Why Asynchronous Rather Than Synchronous, For This Interaction Specifically

This is the interaction where async is *more* appropriate than the sync call used elsewhere in the system (Booking → Equipment availability check), for reasons specific to this case:

1. **The caller does not need the result.** Confirming a booking genuinely depends on knowing whether the equipment is available *right now* - that's a natural synchronous read. Completing a booking does not depend on Equipment Service having already processed the maintenance flag; the booking is complete regardless of whether Equipment Service is up, slow, or momentarily unreachable. 2. **Availability under partial outage.** If Equipment Service is down, a synchronous design would force a choice between failing the booking-completion request (losing the user's action) or silently dropping the maintenance report. With messaging, RabbitMQ durably holds the event until Equipment Service is available again to consume it - the booking completes immediately either way, and the maintenance side effect is never lost. 3. **Decoupled failure domains.** A slow or failing Equipment Service should not make booking completion slow or failing. The sync call (availability check) *should* propagate Equipment Service's health into the booking flow (you cannot confirm a booking you can't verify) - this one deliberately should not. 4. **Natural one-to-many extension point.** A "maintenance requested" event is the kind of fact other future consumers might care about (e.g. a notification service, an analytics service) - a queue/exchange scales to multiple consumers without Booking Service knowing about them; a direct REST call would not.

Alternatives Considered

Alternative Considered	Why Not Chosen
Synchronous REST call from Booking Service to Equipment Service on completion	(mirroring the existing Feign-based availability check). Rejected: couples booking completion's success/latency to Equipment Service's availability for a side effect the caller doesn't need to wait on, and the assignment specifically asks for one interaction to demonstrate messaging instead.
Kafka instead of RabbitMQ.	Considered, but RabbitMQ was chosen for this single point-to-point/topic-routed event: no need for Kafka's log-based replay, partitioning, or consumer-group semantics at this scale, and RabbitMQ's lower operational overhead (single container, simpler `docker-compose` entry, quicker to demonstrate) was a better fit for a two-service, single-event-type system. Empirical message-queue benchmarking supports this: RabbitMQ remains competitive with lower operational complexity at moderate throughput, while Kafka's advantages concentrate at large-scale, high-throughput, multi-consumer-

	group workloads this system does not have [3].
Spring Cloud Stream abstraction over RabbitMQ	instead of raw `spring-boot-starter-amqp` + `RabbitTemplate`/`@RabbitListener`. Rejected for this assignment: Cloud Stream would add a binder abstraction layer that isn't needed for a single exchange/queue and would obscure the exchange/routing-key/queue wiring the report needs to evidence directly.

Consequences

Positive

- Booking completion is fast and does not depend on Equipment Service's availability.
- Equipment Service processes maintenance events at its own pace, and RabbitMQ durably queues them if Equipment Service is temporarily down (queue is declared `durable: true`).
- New consumers of `MaintenanceRequested` can be added without touching Booking Service.

Negative / trade-offs

- Eventual consistency: there is a (normally sub-second) window where a booking is `COMPLETED` but the equipment hasn't yet flipped to `UNDER_MAINTENANCE`. Acceptable for this domain (nobody is blocked waiting on it).
- No compensating transaction/outbox pattern was implemented: if `RabbitTemplate.convertAndSend` fails synchronously (e.g. broker unreachable at publish time) after the booking row is already committed, the event is lost. For this assignment's scope this was accepted as a known simplification rather than implementing a transactional outbox.
- Cross-service type mapping required care: the default `DefaultClassMapper` in `Jackson2JsonMessageConverter` resolves `__TypeId__` by fully-qualified class name, so both services' event classes must share the same package + class name (documented in both `RabbitMQConfig` classes) - a subtlety worth calling out since a mismatch fails silently at consume time with a `MessageConversionException`.

Implementation Evidence

- `booking-service/src/main/java/com/labequip/booking/messaging/MaintenanceEventPublisher.java`
- `booking-service/src/main/java/com/labequip/booking/messaging/RabbitMQConfig.java`
- `equipment-service/src/main/java/com/labequip/equipment/messaging/MaintenanceRequestedListener.java`
- `equipment-service/src/main/java/com/labequip/equipment/messaging/RabbitMQConfig.java`

- `equipment-service/src/main/java/com/labequip/equipment/service/MaintenanceRecordService.java` (`handleMaintenanceRequested`)
- RabbitMQ management UI evidence: `docs/screenshots/02-rabbitmq-queue.jpg`, `docs/screenshots/03-rabbitmq-exchange.jpg`
- Actual message payload (exchange, routing key, JSON body, traceparent header), captured via RabbitMQ's "Get messages" inspector: `docs/screenshots/08-rabbitmq-message-payload.jpg`
- Distributed trace showing the publish/consume hop: `docs/screenshots/05-zipkin-async-trace.jpg` (spans: `booking-service: lab-equipment-exchange/maintenance.requested send` → `equipment-service: maintenance-requested-queue receive`)
- Report evidence: Section 2.3 "Asynchronous Messaging"
- Screencast timestamp: **12:29–13:36**

References

[1] V. Velepucha and P. Flores, "A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges," *IEEE Access*, vol. 11, pp. 88339-88358, 2023, doi: 10.1109/ACCESS.2023.3305687.

[3] R. Maharjan, M. S. H. Chy, M. A. Arju, and T. Cerny, "Benchmarking Message Queues," *Telecom*, vol. 4, no. 2, pp. 298-312, 2023, doi: 10.3390/telecom4020018.

A.3 ADR-003: Resilience and Failure Handling for the Sync Booking → Equipment Call

Status

Accepted

Context

Booking Service must synchronously confirm equipment is `AVAILABLE` before confirming a booking (the assignment's required "Service A must synchronously query Service B before completing at least one operation"). This call crosses a process and network boundary via Eureka + OpenFeign (`EquipmentClient`), so it can fail in ways a plain in-process method call cannot: Equipment Service can be down, slow, or network-partitioned. Without protection, a single slow/failing Equipment Service would make every booking-creation request hang or fail with a raw connection exception, and repeated attempts would keep retrying against a service that isn't coming back soon - wasting threads/connections and degrading Booking Service itself.

Decision

The call is wrapped in `EquipmentAvailabilityService.fetchEquipment()` (`booking-service/.../service/EquipmentAvailabilityService.java`) with two Resilience4J annotations, configured per-instance as `equipmentService` in Config Server (`config-server/.../config/booking-service.yml`):

```
resilience4j:
  circuitbreaker:
    instances:
      equipmentService:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
  retry:
    instances:
      equipmentService:
        max-attempts: 3
        wait-duration: 500ms
feign:
  client:
    config:
      equipment-service:
        connect-timeout: 2000
        read-timeout: 2000
```

- `**@Retry**` absorbs transient blips (e.g. a single dropped connection) with 3 attempts, 500ms apart - short enough not to make the caller wait unreasonably long.
- **Feign connect/read timeouts (2s)** bound how long a single attempt can hang, so a slow (not down) Equipment Service can't tie up a Booking Service thread indefinitely.
- `**@CircuitBreaker**` tracks the last 10 calls; once at least 5 calls have been made and 50%+ failed, it opens for 10 seconds, during which calls fail immediately (`notPermittedCalls`, verified in testing: see Implementation Evidence) without attempting the network call at all - protecting Booking Service from wasting resources on a service known to be down. It then half-opens automatically to test recovery with a limited number of trial calls. This CLOSED → OPEN → HALF_OPEN lifecycle mirrors circuit-breaker refinements proposed to reduce state-transition delay and improve failure detection in microservice architectures [2].
- `**fallbackMethod = "equipmentUnavailableFallback"**` runs whenever the retries are exhausted or the circuit is open, and throws a `ResponseStatusException(503, ...)` with a clear message - turning an opaque connection failure into a well-formed API response the client can act on, rather than a hung request or a raw 500.

Alternatives Considered

Alternative Considered	Why Not Chosen
No resilience wrapper - let the Feign call fail/hang naturally.	Rejected outright: violates the assignment's explicit resilience requirement and would mean any Equipment Service blip directly produces hung requests or ugly 500s at the Gateway.
Retry only, no circuit breaker.	Would absorb transient failures but keeps hammering a genuinely-down dependency on every subsequent request, each still paying the full retry+timeout cost. Rejected: doesn't protect Booking Service's own resources under a sustained outage, which is the more realistic multi-minute failure mode a circuit breaker specifically addresses.
Circuit breaker only, no retry.	Rejected: without a retry, a single transient TCP hiccup (not indicative of Equipment Service actually being down) would count as a hard failure and contribute to tripping the breaker unnecessarily; retry lets genuinely transient failures self-heal before they're counted against the breaker's failure budget.
`@TimeLimiter` (async, `CompletableFuture`-based) in addition to circuit breaker/retry.	Considered for stricter latency bounding, but rejected in favour of the simpler Feign-level connect/read timeouts: `@TimeLimiter` requires the wrapped method to return `CompletableFuture`, which would have pushed the synchronous booking-creation flow onto an async model for no benefit, since Feign's own timeout already bounds worst-case latency per attempt.
****Fail the booking silently / return the equipment as unavailable rather than 503 on**	circuit-open.** Rejected: conflating "equipment is in use" (a real business state, `409`) with "we couldn't find out" (an infrastructure failure, `503`) would mislead the client into thinking the equipment itself was the problem rather than the dependency being unreachable.

Consequences

Positive

- Booking Service degrades gracefully under Equipment Service outages: callers get a fast, clear `503` (confirmed in testing at ~1-1.3s, versus a multi-second/indefinite hang without the timeout+circuit-breaker combination) instead of a hang or raw exception.
- The circuit breaker prevents pointless repeated network attempts against a service that's known to be down for the `wait-duration-in-open-state` window, then automatically probes recovery.
- Distinct HTTP semantics: `409` for "equipment genuinely not available" vs `503` for "couldn't verify due to a dependency failure" - a real business state is not confused with an infrastructure failure.

Negative / trade-offs

- Adds a fixed worst-case failure-path latency of up to roughly `3 attempts × ~2s timeout` before the circuit trips open; once open, subsequent calls fail fast. This was judged an acceptable trade-off given the assignment's small scale and the value of not treating one bad probe as a full open.
- The fallback treats *any* failure the same way (503, "could not verify"), including a 404 from Equipment Service for a genuinely non-existent equipment ID - which is also counted as a circuit breaker failure. In a production system this would likely be refined with `ignoreExceptions` so that legitimate 4xx business responses don't count against the breaker's failure budget; this was accepted as a known simplification for the assignment's scope.
- Resilience4J configuration lives in Config Server rather than code, which is consistent with the externalised-configuration requirement but means tuning it requires a config refresh/restart rather than a code change.

Implementation Evidence

- `booking-service/src/main/java/com/labequip/booking/service/EquipmentAvailabilityService.java`
- `booking-service/src/main/java/com/labequip/booking/client/EquipmentClient.java`
- `config-server/src/main/resources/config/booking-service.yml` (`resilience4j.*`, `feign.client.config`)
- Resilience evidence: `docs/screenshots/11-resilience.jpg` - circuit breaker state `CLOSED` → repeated `503`s → `OPEN` (`failureRate: 80%`, `notPermittedCalls: 17`) → automatic `HALF_OPEN` transition (raw transcript also kept at `docs/evidence-resilience.txt`)
- Report evidence: Section 2.3 "Service-to-Service Communication and Resilience"

- Screencast timestamp: **11:34–12:29**

References

[2] A. Hlybovets and I. Paprotskyi, "Increasing the Fault Tolerance in Microservice Architecture," *Cybernetics and Systems Analysis*, vol. 60, no. 3, pp. 480-488, 2024, doi: 10.1007/s10559-024-00689-0.